

Module	ET4047 (Embedded Software)
Date	10/09/2026
Lab No	Lab 1 – Wokwi Simulation Primer
Author	Luke Griffin

1 Objectives

- ✓ Introduce students to the **Wokwi** online circuit simulator as a zero-hardware development environment.
- ✓ Learn how to create a project, add components, and wire a circuit on the virtual canvas.
- ✓ Write, compile, and run a simple Arduino sketch and observe the result in the simulation.
- ✓ Relate individual lines of code to the observable behaviour of the circuit.
- ✓ Build confidence with the edit–run–observe workflow before moving to physical hardware.

2 Getting Started with Wokwi

Wokwi is a browser-based simulator that models both the microcontroller and the components attached to it, running your real firmware against a virtual circuit. Because nothing is physical, there is no risk of damaging hardware and no need to install a toolchain to get started.

Open a browser and go to Wokwi, then create a new project:

<https://wokwi.com>

1. On the home page, select **Start a New Project**.
2. Choose the **Arduino Nano** template. (Wokwi does not simulate the Nano Every used elsewhere in this module; the Nano shares the same Arduino C interface and is the reliable target for this primer.)
3. You are taken to the workspace: the **diagram canvas** on the left and the **code editor** on the right.

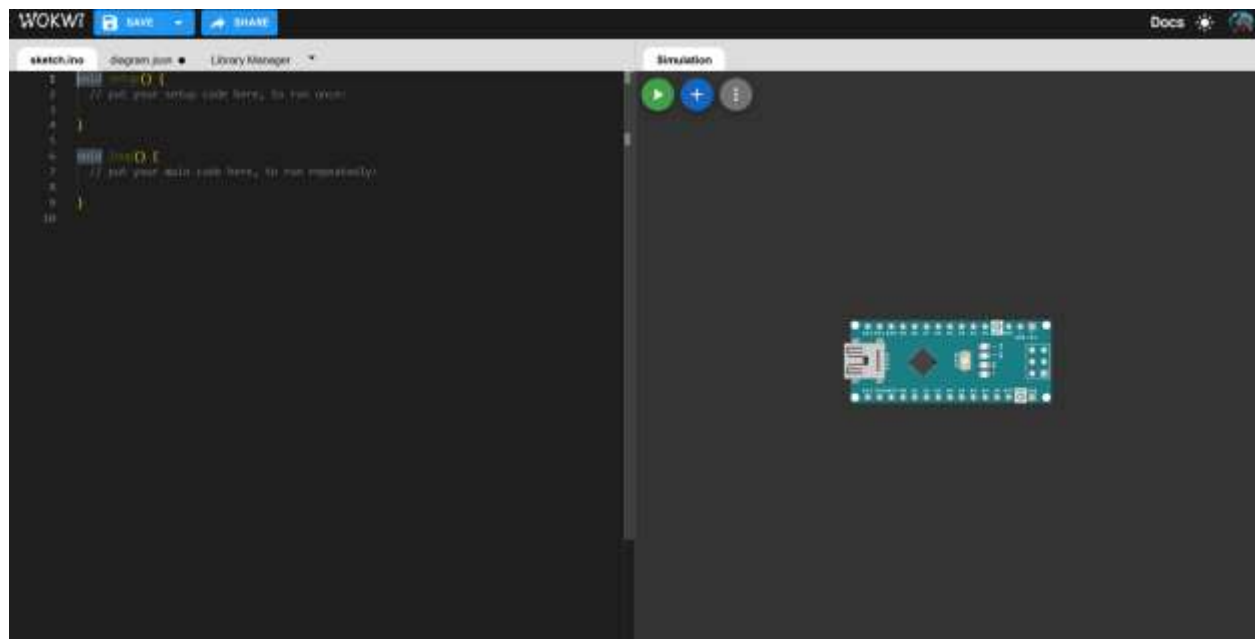


Figure 1: The empty Wokwi workspace – Arduino Nano and code editor

3 The Wokwi Workspace

Two areas do all the work. The **diagram canvas** is where you place and wire components; click the **+** button to open the parts library and search for parts such as LEDs, resistors, buttons, and potentiometers. The **code editor** is where you write the sketch that runs on the virtual board. The green **Start Simulation** button runs everything; the red button stops it and any errors appear beneath the editor.

4 Lab Tasks

In this lab you will build the classic first circuit of embedded systems – a blinking LED – and then modify it to see how the code drives the behaviour.

1. **Blinking LED** – Digital I/O, configuring an output pin, timing with `delay()`.

Add an **LED** and a **220 Ω resistor** from the parts library. Wire the LED's longer leg (anode) to digital **pin 13**, its shorter leg (cathode) through the resistor to a **GND** pin. Then enter the sketch below and press Start Simulation.

```
// Lab1_Task1 - Blink an LED on pin 13
// Author: <your name>

#define LED_PIN 13          // output pin

void setup() {
  pinMode(LED_PIN, OUTPUT); // configure pin as an output
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // LED on
}
```

```
    delay(1000);           // wait 1000 ms
    digitalWrite(LED_PIN, LOW); // LED off
    delay(1000);           // wait 1 second
}
```

Questions?

- » What happens to the blink rate if you change both `delay(1000)` values to `delay(200)`?
- » What does the `pinMode()` line do, and what happens to the LED if you remove it?

Simulation Output

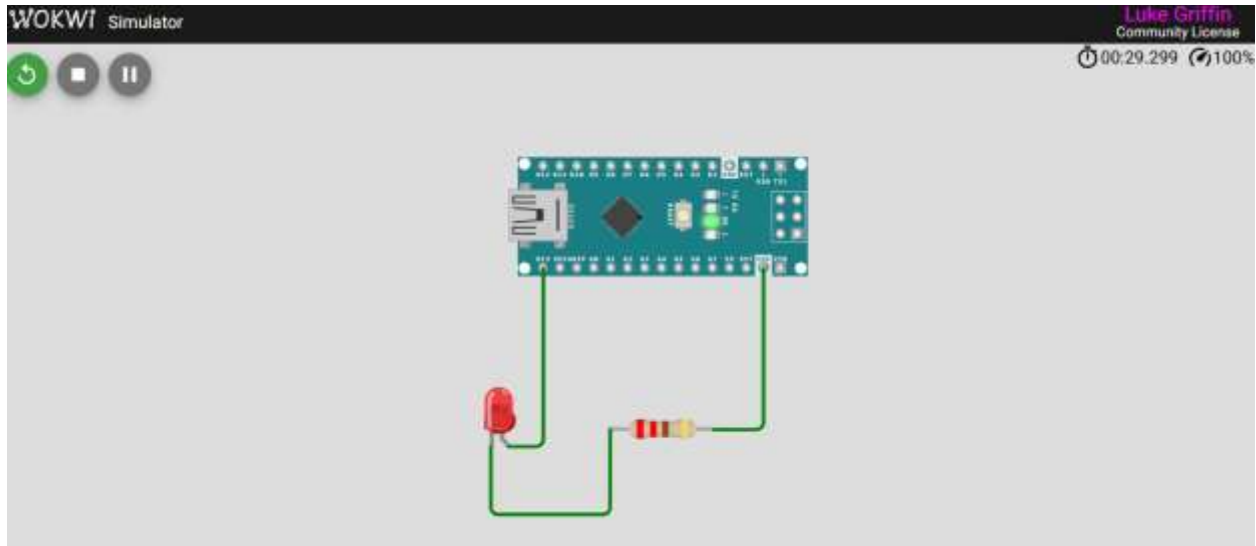


Figure 2: LED circuit running in Wokwi

2. **Software PWM** – on/off toggling to dim an LED, timing with `millis()`.

Keep the same circuit. Replace the sketch with the one below, which switches the pin on and off far faster than the eye can follow so the LED appears dimmed. Change `DUTY_PCT` and observe the brightness.

```
// Lab1_Task2 - Software PWM on a digital pin using millis()
// Author: <your name>

#define PWM_PIN    13           // output pin
#define PERIOD_MS  200UL        // full cycle in ms
#define DUTY_PCT   25           // duty cycle in percent (0..100)

const unsigned long on_time_ms = (PERIOD_MS * DUTY_PCT) / 100;
unsigned long start_ms = 0;

void setup() {
    pinMode(PWM_PIN, OUTPUT);
    Serial.begin(115200);        // open Serial Monitor / Plotter
    start_ms = millis();
}

void loop() {
    unsigned long now = millis();
    unsigned long phase = (now - start_ms) % PERIOD_MS;
```

```
int pwm_state = (phase < on_time_ms) ? HIGH : LOW;  
digitalWrite(PWM_PIN, pwm_state);  
Serial.println(pwm_state); // view in the Serial Plotter  
}
```

Questions?

- » How does the LED's brightness change as you raise or lower DUTY_PCT?
- » Open the Serial Plotter. What shape does the plotted signal take, and why?

Serial Plotter Output

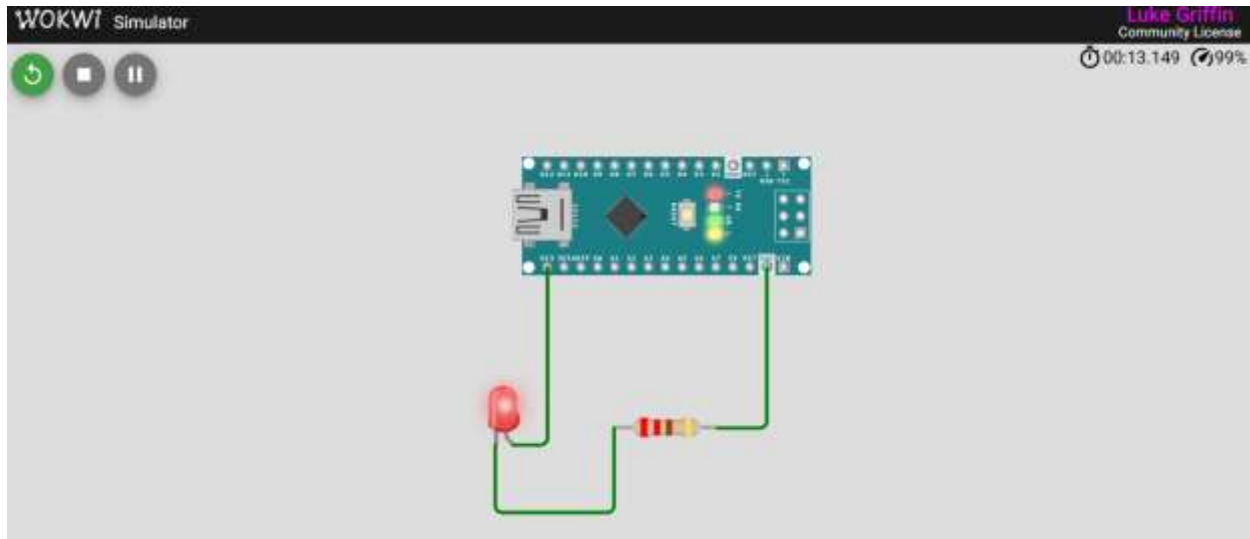


Figure 3: Software-PWM square wave in the Serial Plotter

3. Potentiometer read – ADC (analog-to-digital conversion), scaling, Serial.print().

Add a **potentiometer** from the parts library. Wire its two outer pins to **5V** and **GND**, and its wiper (centre pin) to **A0**. Run the sketch, open the Serial Monitor, and turn the knob.

```
// Lab1_Task3 - Read a potentiometer and print to Serial  
// Author: <your name>  
  
#define POT_PIN    A0  
#define VREF_MV    5000UL // mV reference  
  
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  int raw = analogRead(POT_PIN); // 0..1023  
  unsigned long mv = (unsigned long)raw * VREF_MV / 1023UL;  
  Serial.print("raw: ");  
  Serial.print(raw);  
  Serial.print(" mv: ");  
  Serial.println(mv);  
  delay(100);  
}
```

Questions?

- » What is the range of the `raw` value, and why is it that range for a 10-bit ADC?
- » What happens to the printed output if you change the baud rate in the Serial Monitor but not in the code?

Serial Monitor Output

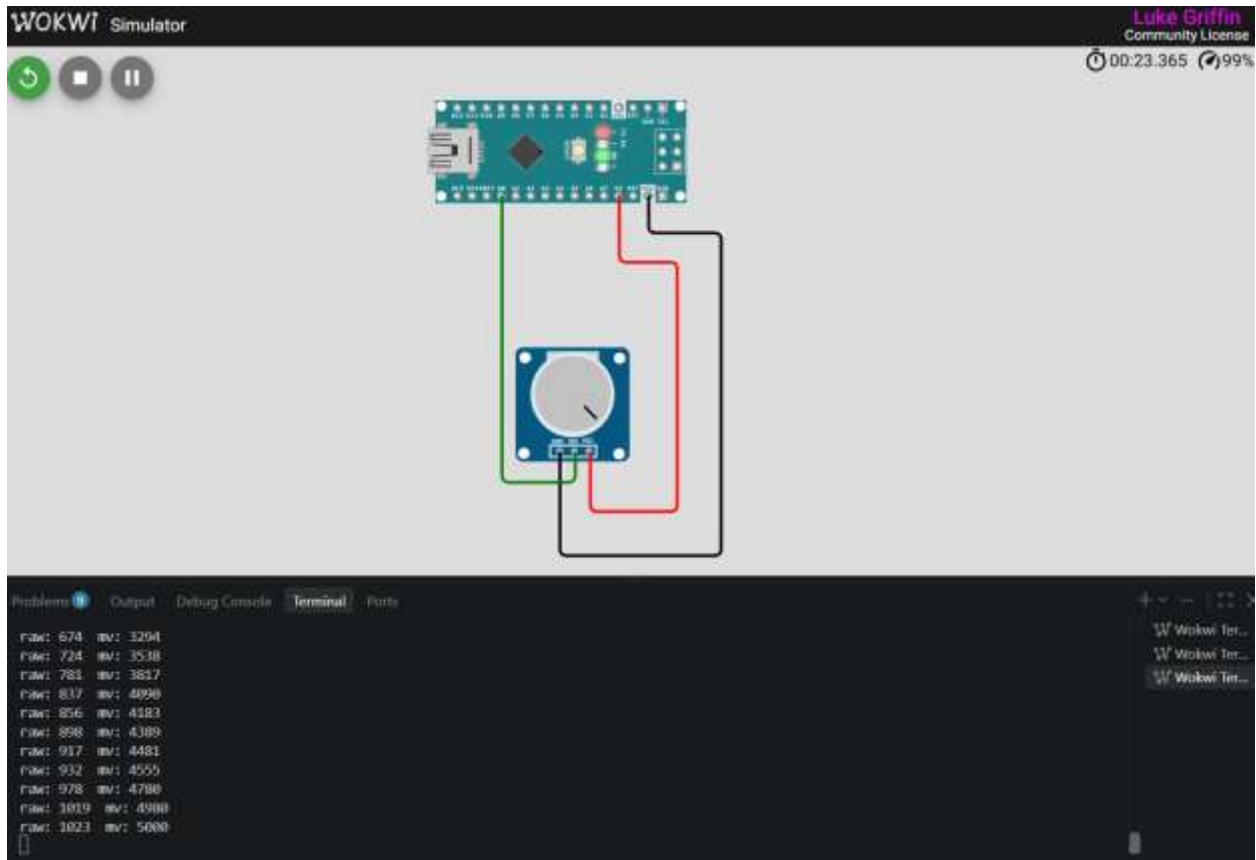


Figure 4: Potentiometer values printed to the Serial Monitor